week02 报告

1. 项目前后端启动测试

在仓库根目录开启四个终端：

- 终端1：启动数据库

```
docker compose up -d mysql redis
powershell -ExecutionPolicy Bypass -File .\scripts\migrate.ps1
```

- 终端2：API

```
cd syncflow\backend
.\.venv\Scripts\Activate.ps1
$env:UPLOAD_DIR = "E:\Project\syncflow\uploads"
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

```
syncflow on  main [?⇡]
❯ cd E:\Project\syncflow\backend

syncflow\backend on  main [?⇡] via 🐍
v3.13.2
❯ .\.venv\Scripts\Activate.ps1

syncflow\backend on  main [?⇡] via 🐍
v3.13.2 (.venv)
❯ $env:UPLOAD_DIR = "E:\Project\syncflow\uploads"

syncflow\backend on  main [?⇡] via 🐍
v3.13.2 (.venv)
❯ uvicorn app.main:app --reload --host
127.0.0.1 --port 8000
INFO:     Will watch for changes in these directories: ['E:\\Project\\syncflow\\backend']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [58808] using WatchFiles
INFO:     Started server process [49116]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:40361 - "GET /api/v1/jobs?page=1&page_size=20 HTTP/1.1" 200 OK
INFO:     127.0.0.1:40363 - "GET /api/v1/jobs?page=1&page_size=20 HTTP/1.1" 200 OK
INFO:     127.0.0.1:40369 - "GET /healthz HTTP/1.1" 200 OK
```

运行API

- 终端3：Worker

```
cd E:\Project\syncflow\backend
.\.venv\Scripts\Activate.ps1
python -m app.worker
```

```
syncflow on  main [?⇡]
❯ cd E:\Project\syncflow\backend

syncflow\backend on  main [?⇡] via 🐍 v3.13.2
❯ .\.venv\Scripts\Activate.ps1

syncflow\backend on  main [?⇡] via 🐍 v3.13.2 (.venv)
❯ python -m app.worker
2026-10-08 10:25:28,755 INFO service=worker worker started
2026-10-08 10:27:07,277 INFO service=worker job_id=9b41f9df-a1e3-4c1a-be0f-7e8c8b6ea240 status=RUNNING
2026-10-08 10:27:07,286 INFO service=worker job_id=9b41f9df-a1e3-4c1a-be0f-7e8c8b6ea240 status=SUCCESS
2026-10-08 10:28:31,071 INFO service=worker job_id=576ee2ac-8e98-4ad8-a773-4d8140a6a293 status=RUNNING
2026-10-08 10:28:31,091 INFO service=worker job_id=576ee2ac-8e98-4ad8-a773-4d8140a6a293 status=SUCCESS
```

运行Worker截图

- 终端4：前端页面

```
cd E:\Project\syncflow\frontend
npm run dev
```

```
syncflow on  main [?⇡]
❯ cd E:\Project\syncflow\frontend

syncflow\frontend on  main [?⇡] via
v22.14.0
❯ npm run dev

> frontend@0.0.0 dev
> vite


  VITE v8.3.1  ready in 548 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: http://100.122.158.67:5173/  Tailscale
  ➜  Network: http://192.168.5.4:5173/
    Ethernet
  ➜  Network: http://192.168.64.1:5173/    VMware Network Adap…
  ➜  Network: http://192.168.226.1:5173/   VMware Network Adap…
10:33:37 AM [vite] (client) hmr update /src/components/CreateJobForm.tsx
```

运行前端截图

完成上述四个步骤后，先测试API：

```
curl.exe http://127.0.0.1:8000/healthz
```

如果返回 "status": "ok"，则启动成功，然后在浏览器中打开 http://127.0.0.1:5173 。在根目录做一份CSV：

```
cd E:\Project\syncflow
@'
external_id,name,amount,record_date
S001,Goods A,19.90,2026-01-01
'@ | Set-Content -Encoding utf8 .\sample.csv
```

1 打开首页。没有任务时会看到空状态和「创建任务」。

2 点击创建任务，名称填 商品导入，文件选 sample.csv，提交。列表里出现这条任务，地址栏是 http://127.0.0.1:5173/。

3 查看终端 3。日志里会有这个任务的 job_id，状态从 RUNNING 变成 SUCCESS。刷新页面后，状态文字是已完成。uploads 里有一个 {任务id}.csv，文件名不是你上传时的原名。

4 状态下拉选已完成。地址栏变成 /?status=SUCCESS。再选「等待中」，这条任务不在列表里，地址栏是 /?status=PENDING。选回「全部」会去掉 status。

5 点击该任务的详情。页面显示源文件名 sample.csv、总数 0、成功 0、失败 0，以及创建、开始、结束时间，格式类似 2026-10-08 09:37:08。最近错误是「无」。点「返回列表」回到首页。

6 再创建一次，这次选一个 .txt 文件。表单上出现错误，列表里不会多出任务。

7 地址栏改成 http://127.0.0.1:5173/jobs/not-a-real-id ，页面显示「任务不存在」，可以返回列表。

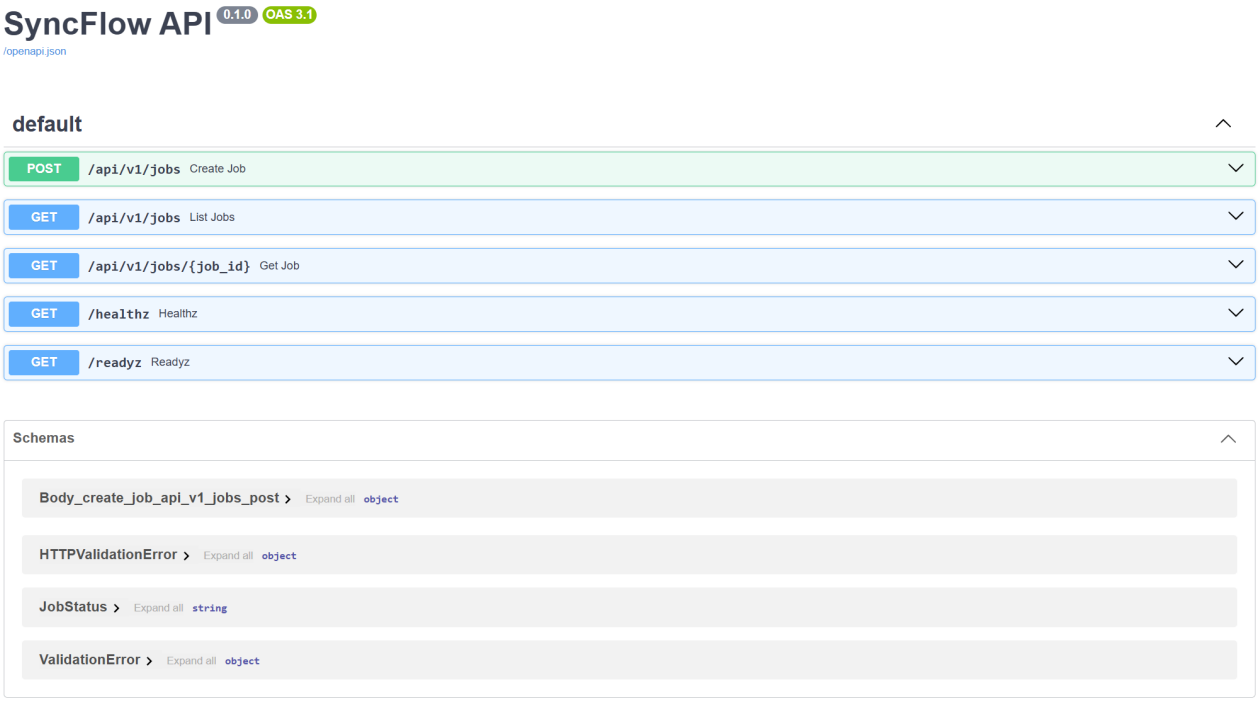
8 地址栏改成 http://127.0.0.1:5173/no-such-page ，页面显示「页面不存在」。

2. 项目接口文档

接口文档由FastAPI自动生成。API 在 http://127.0.0.1:8000 运行时，有三个地址：

- http://127.0.0.1:8000/docs 是可交互的 Swagger 页面，可以直接试请求。
- http://127.0.0.1:8000/redoc 是只读的接口说明。
- http://127.0.0.1:8000/openapi.json 是 OpenAPI 规范原文。

创建、列表和详情这三条路径都在这份规范里。



项目接口文档

3. 数据库初始化与迁移

MySQL 容器第一次初始化空数据卷时，只会创建空库 syncflow 和用户 syncflow。sync_jobs、sync_records、sync_errors 不在镜像里，由 scripts/init_db.sql 建立。Windows 执行 scripts/migrate.ps1，Linux 执行 scripts/migrate.sh。脚本读取仓库根目录的 .env，再把 SQL 送进已经运行的 MySQL 容器。

日常开发迁移开发库：

```
powershell -ExecutionPolicy Bypass -File .\scripts\migrate.ps1
```

成功时终端打印 migrated: syncflow。

三张表都使用 InnoDB 和 utf8mb4。时间字段是 DATETIME(3)，存的是 UTC。表之间不建外键，关联由应用维护。

表	作用	关键约束
sync_jobs	一次 CSV 导入任务	主键 id 为 VARCHAR(36)。索引 (status, created_at) 和 (created_at)，给列表的状态筛选和按创建时间倒序用
sync_records	校验通过后的成功记录	金额是 DECIMAL(12,2)。UNIQUE(job_id, external_id)，保证同一任务里的业务标识不重复
sync_errors	校验失败的错误记录	索引 (job_id, row_number)。原始行摘要用 JSON

本周 Worker 只把任务从 PENDING 改为 RUNNING，再改为 SUCCESS，还不解析 CSV，所以不会往 sync_records 和 sync_errors 插入数据。这两张表先建好，留给后面写入。

脚本可以重复执行。建表语句都是 CREATE TABLE IF NOT EXISTS，同一份 SQL 跑两遍，表还在，也不会因为表已存在而失败。

开发库和测试库分开。页面和接口平时连 syncflow。backend/tests 里的集成测试连 syncflow_test，避免测试数据写进开发库。MySQL 镜像不会自动创建测试库，业务账号也不能 CREATE DATABASE。测试库要用：

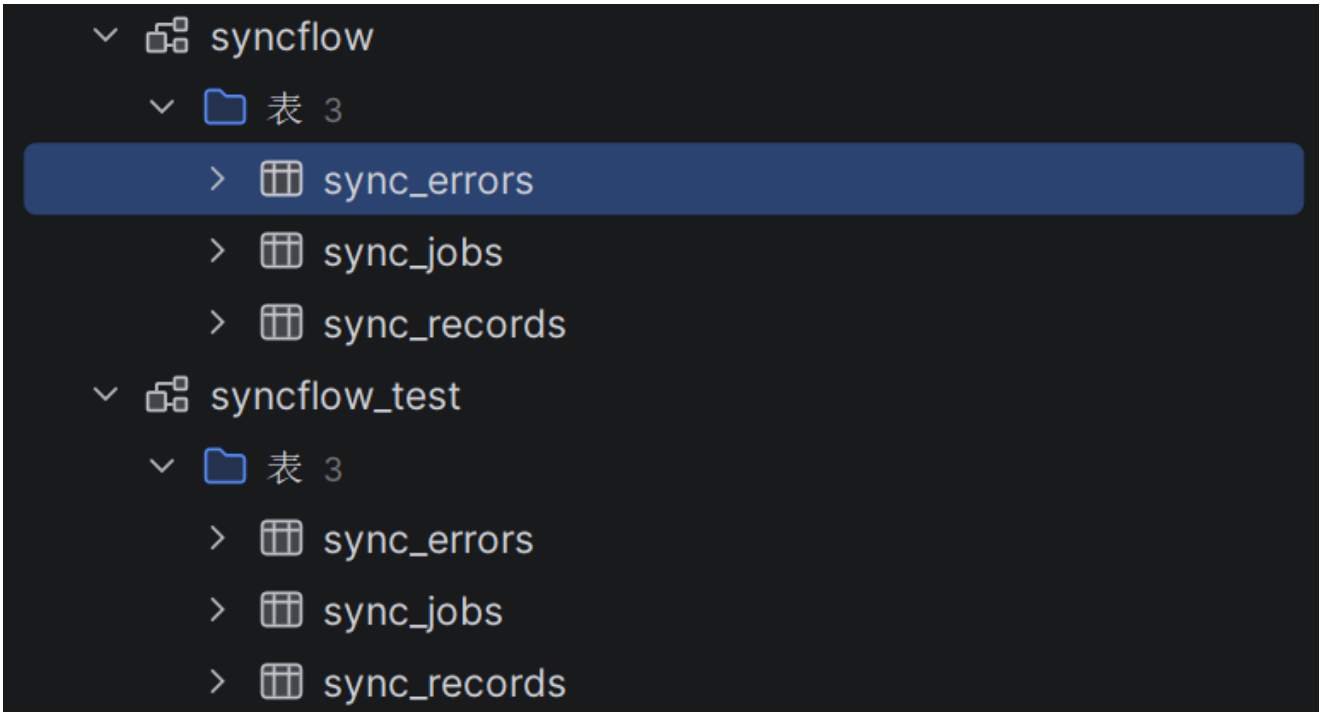
```
powershell -ExecutionPolicy Bypass -File .\scripts\migrate.ps1 -Test
```

这条命令先用 root 执行 CREATE DATABASE IF NOT EXISTS，把 syncflow_test 的权限授给用户 syncflow，再用同一份 init_db.sql 建表。应用从 MYSQL_HOST、MYSQL_PORT、MYSQL_USER、MYSQL_PASSWORD、MYSQL_DATABASE 拼出连接串；测试可以通过 MYSQL_TEST_DATABASE 连到另一个库。

建完后可以在根目录核对：

```
docker compose exec -T mysql mysql --user=$env:MYSQL_USER --password=$env:MYSQL_PASSWORD
--host=127.0.0.1 -e "SHOW TABLES FROM syncflow; SHOW TABLES FROM syncflow_test;"
```

两个库里都应有且只有 sync_jobs、sync_records、sync_errors。



数据库截图